IU · PROJECT: SOFTWARE ENGINEERING (CSEMCSPSE01) · PORTFOLIO PART 3

Tulsi Tiffins POS

Project Management Report

Plan against actual, milestones and releases, scope, risk, quality and configuration management, and the index of all submitted documents.

Author	**Praveen Mysore Jagannath** · 10251728
Tutor	Niknejad, Naghmeh
Document version	**1.0.0** (matches software release v1.0.0)
Date	28 October 2026
Project plan	Phase 1 report v0.1.0, Section 5 (Table 7, Gantt chart)
Source code	github.com/mjpraveen04/tulsi-pos

Contents

1 Project profile in brief

2 Schedule – plan against actual

3 Milestones and releases

4 Scope management

5 Risk management

6 Quality management

7 Configuration and release management

8 Resources and budget

9 Index of submitted documents

1 Project profile in brief

The full project profile – background, objective, scope, target users, organisation and risks – is the Phase 1 report (`MysoreJagannath-Praveen_10251728_PSE_P1_S.pdf`, in this folder). This report records how the project was actually run against that profile.

Item	Phase 1 baseline
Objective	One affordable, touch-friendly web application that unifies register, inventory, orders, staff and payments for a small café, so that every sale updates stock correctly and is recorded with a full audit trail.
Scope (MVP)	PIN login with two roles; staff management; products and an append-only stock ledger; a Register that cannot oversell; atomic billing; cash and UPI (static QR); searchable order history.
Out of scope	UPI gateway with webhooks, supplier purchase orders, analytics dashboards, table and reservation management (FR-F1…F4).
Methodology	Waterfall: conception, design, implementation, testing, deployment; requirements frozen with MoSCoW before coding.
Organisation	Single developer in all roles – project lead, requirements analyst, developer, tester, technical writer; tutor review at the end of each phase.
Planned end	25 September 2026
Budget	No funding: free and open-source tools, free hosting tiers.

2 Schedule – plan against actual

The plan had nine work packages from 4 August to 25 September 2026. The implementation and test work packages WP1–WP8 were finished early, by 13 August, for the Phase 2 submission. Deployment and final documentation (WP9) were planned for 20–25 September but were done on 26–28 October, after a break of about two months with no development. The release therefore came **33 days after the planned end**.

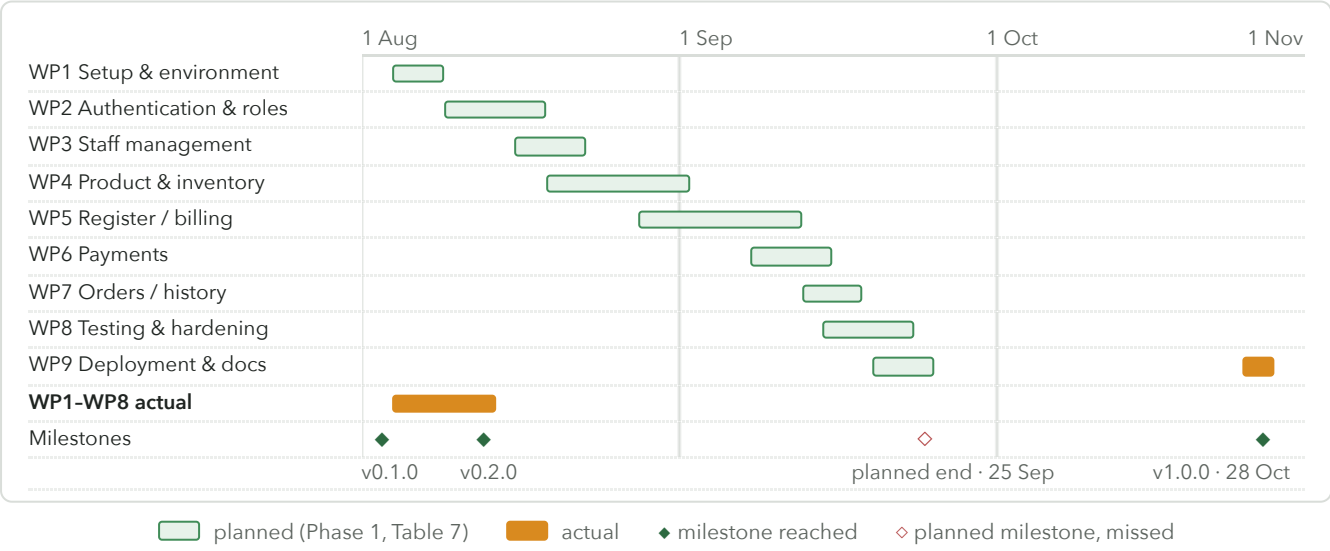


Figure 1: Work packages, planned against actual, August – October 2026 Gantt chart · legend above

Work package	Planned	Actual	Result and deviation
WP1 Setup & environment	04 – 08 Aug	4 – 13 Aug (not tracked per package)	Docker Compose, Alembic, project skeleton.
WP2 Authentication & roles	09 – 18 Aug		PIN login, JWT, server-side roles.
WP3 Staff management	16 – 22 Aug		Add, reset PIN, deactivate; last-manager guard added.
WP4 Product & inventory	19 Aug – 01 Sep		Products, stock ledger, low-stock badges; photo upload added.
WP5 Register / billing	28 Aug – 12 Sep		Atomic sale with row locks; bill number after the stock check.
WP6 Payments	08 – 15 Sep		Cash with change, static UPI QR.
WP7 Orders / history	13 – 18 Sep		Bill history, receipt view, daily total.
WP8 Testing & hardening	15 – 23 Sep		118 tests incl. 6 release gates; each module tested when finished. **About 6 weeks ahead of plan.**
WP9 Deployment & documentation	20 – 25 Sep	26 – 28 Oct	Strict TypeScript, GitHub, Render + Neon deployment, photos in PostgreSQL, bill-number fix, UML documentation, manuals. **33 days late.**

> **Why the actual dates of WP1–WP8 are not known in detail:** version control was set up only after the implementation – the first commit is from 23 August. Dates per work package were therefore not recorded and are not reconstructed here. Committing from the first day is the first lesson in Section 7.

3 Milestones and releases

Date	Version	Milestone	Content
3 Aug 2026	v0.1.0	Phase 1 – conception submitted	Project profile, MoSCoW requirements, use case and domain model, C4 context and container views, Gantt plan, test cases.
13 Aug 2026	v0.2.0	Phase 2 – development submitted	All 30 functional requirements implemented, migration 0001, 118 tests on PostgreSQL.
23 Aug 2026	–	Repository published	First commit to github.com/mjpraveen04/tulsi-pos; the design hand-off was removed from the public repository.
26 Oct 2026	v0.9.0	Phase 2 presentation revised	Front end migrated to strict TypeScript; presentation deck for the tutor review.
27 Oct 2026	–	Tutor feedback received	Three points: rationale against alternatives, notation on every diagram, more specific lessons learned.
28 Oct 2026	**v1.0.0**	**Phase 3 – final release**	Cloud deployment, photos in PostgreSQL (migration 0002), short-cash fix, 120 tests, UML 2.5 architecture documentation, requirements traceability, installation manual. Git tag `v1.0.0`.

4 Scope management

The scope was fixed in Phase 1 with MoSCoW. The rule was that a new idea is logged for a later phase, never built mid-project; it was also written into the repository's build rules (`CLAUDE.md`, "Scope discipline").

Measure	Result
Won't-have items built	**0 of 4** – no UPI gateway, purchase orders, charts or table management.
Requirements dropped	**0 of 30** functional, 0 of 18 non-functional.
Requirements delivered	29 of 30 functional requirements fully met (FR-16: 2-second target not measured); 14 of 18 non-functional met.
Changes during the project	Nine refinements, all additive – for example short-cash rejection, the last-manager guard and photo storage in the database. Listed with reasons in the Requirements and Traceability Report, Section 7.
Work cut when time ran short	The planned load test (NFR-01…03) – recorded as technical debt TD-01.

5 Risk management

The risk register from Phase 1 (Tables 3 and 4) was reviewed at the end of the project. The detailed evidence for each risk is in the Requirements and Traceability Report, Section 6.

ID	Risk	Status	What happened
TR-01	Race condition oversells	Closed	Row locks and CHECK; concurrency release gates pass.
TR-02	Partial writes	Closed	One transaction; fault-injection gate passes.
TR-03	Prototype does not translate to React	Closed	Prototype used as visual reference only.
TR-04	Rounding errors	Closed	Decimal throughout the server; browser preview noted as TD-09.
TR-05	Different behaviour on another machine	Mitigated	Same images locally and in the cloud; fresh-clone install verified; PostgreSQL 16 vs 18 (TD-08).
BR-01	Scope creep	Closed	MoSCoW freeze held.
BR-02	Owners distrust a self-hosted till	Open	Not validated with a real café.
BR-03	Solo timeline stalls	Occurred	Two months without development; release 33 days late. Must-haves were front-loaded, so the impact was limited to the load test and the deployment date.
BR-04	A gap is found late	Partly occurred	Short cash used up a bill number, and the cloud host would have lost photos; both found and fixed in the final phase.

6 Quality management

Measure	How it was applied
Definition of done per work package	Endpoints match the API contract including status codes; the package's tests pass; the concurrency and rollback gates stay green; `docker compose up` still gives a working application (`CLAUDE.md`).
Automated tests	120 tests on real PostgreSQL, run before every commit; release gates as a separate marker (`-m gate`).
Test-first bug fixes	Every defect found was first reproduced by a failing test, e.g. the bill-number gap (1044 instead of 1043).
Static checks	Strict TypeScript; `npm run build` fails on any type error, including a mismatch with the API types.

Measure	How it was applied
Reviews	Code-structure review for the layering rule; schema review for audit columns; tutor review after each phase.
Acceptance	Test cases TC-01…08 automated; live smoke test on the hosted application on 28 Oct 2026; fresh-clone installation test.

7 Configuration and release management

- **Repository:** one public GitHub repository with code, Docker and cloud configuration, tests and all documentation (`docs/`). Work is committed directly to `main` – appropriate for a single developer – and every push to `main` redeploys the hosted application.
- **Versioning:** semantic versions. The documents carry the same version as the software; v1.0.0 is marked by the git tag `v1.0.0` and a GitHub release, which freeze code, diagrams and documents together.
- **Database schema:** versioned Alembic migrations (0001, 0002); never changed by hand.
- **Diagrams as code:** PlantUML sources in `docs/architecture/diagrams/`, rendered to PNG for each release.
- **Submission copy:** the code zip in `04–Implementation` is generated from the tag with `git archive v1.0.0`, so it is identical to the GitHub repository.

> **Lesson:** git was introduced only after the implementation, so the versions v0.1.0 and v0.2.0 exist as document versions but not as commits or tags. In a next project the repository would be created on day one and every phase tagged.

8 Resources and budget

Resource	Used	Cost and limits
People	One developer, part-time beside other commitments	The main constraint of the project (BR-03).
Development tools	Python, FastAPI, PostgreSQL, React, TypeScript, Docker, PlantUML, pytest	Free and open source.
Code hosting	GitHub (public repository)	Free.
Application hosting	Render (web service and static site)	Free plan; API sleeps after 15 minutes, first request 30–60 s (TD-05).
Database hosting	Neon PostgreSQL	Free plan without expiry; chosen because Render's free database is deleted after 30 days.
Coding assistance	AI coding assistant, guided by the written build rules in `CLAUDE.md`	Every generated change was checked by the test suite.
Total cost	**€0** – the budget constraint was met.	

9 Index of submitted documents

Folder	File	Content
Main folder	MysoreJagannath-Praveen_10251728_PSE_P3_S.pdf	2-page abstract (making of, lessons learned)
	MysoreJagannath-Praveen_10251728_PSE_P3_S.txt	GitHub link, application link, login data
01-Project-management	MysoreJagannath-Praveen_10251728_PSE_P1_S.pdf	Result of Phase 1: project profile, plan, requirements, glossary, design
	Tulsi-Tiffins-POS_Project-Management-Report_v1.0.0.pdf	This report
02-Requirements	Tulsi-Tiffins-POS_Requirements-and-Traceability_v1.0.0.pdf	Requirements status, traceability to code and tests, glossary
03-Architecture-documentation	Tulsi-Tiffins-POS_Architecture-Documentation_v1.0.0.pdf	arc42 architecture, technology decisions, ADRs, technical debt
	diagrams/ (PlantUML sources and PNG)	Eight UML 2.5 diagrams
	Phase 2 result (presentation and summary)	Result of Phase 2, as submitted and revised after feedback
04-Implementation	MysoreJagannath-Praveen_10251728_PSE_Submission_Code.zip	All files of the GitHub repository at tag v1.0.0
	Tulsi-Tiffins-POS_Installation-Manual_v1.0.0.pdf	Installation and operating manual